

인터뷰 키트 · 주문·재고 API v2

제출 [@ d35969b619823452e5edc9bf23ed75dc8f596219](https://github.com/wanted-hack-inchyangv/seojin-order-api)

기준 버전 v2-be07fb44

워크벤치 <https://ohmyti.vercel.app/evaluations/1bc69b13-00c6-4283-aa1b-52e6f5616362?tab=questions>

질문 9개 · 필수 3개 · 권장 3개 · 선택 3개

이력서 연결 질문에는 지원자 이력서의 인용이 들어 있습니다. 복사한 내용은 면접 담당자에게만 공유해 주세요.

45분 진행안 구간 합 45분

도입 5분	
설계 트레이드오프 6분	Q4 권장 · 설계 트레이드오프 · 6분
강점 확인 12분	Q1 필수 · 강점 확인 · 6분 Q2 필수 · 강점 확인 · 6분
요구사항 확장 12분	Q3 필수 · 요구사항 확장 · 6분 Q5 권장 · 요구사항 확장 · 6분
이력서 연결 5분	Q6 권장 · 이력서 연결 · 5분
지원자 질문 5분	

60분 진행안 구간 합 45분

도입 5분	
설계 트레이드오프 6분	Q4 권장 · 설계 트레이드오프 · 6분
강점 확인 12분	Q1 필수 · 강점 확인 · 6분 Q2 필수 · 강점 확인 · 6분
요구사항 확장 12분	Q3 필수 · 요구사항 확장 · 6분 Q5 권장 · 요구사항 확장 · 6분
이력서 연결 5분	Q6 권장 · 이력서 연결 · 5분
지원자 질문 5분	

Q1 필수 강점 확인 경계 조건과 실패 처리 · 6분

R-03 재고 부족 거절이 기대대로 동작했습니다. 재고와 요청 수량을 비교하는 판단을 코드의 어느 지점에 두셨나요?

의도 통과한 재고 검증 동작을 스스로 설명할 수 있는지, 판단 위치를 짚을 수 있는지 확인한다.

꼬리 질문

1. 재고 확인과 차감 사이의 순서는 어떻게 되나요?
2. 재고가 0인 상품과 수량이 재고를 넘는 경우를 같은 경로로 처리하셨나요?
3. 이 판단을 다른 계층으로 옮기면 무엇이 달라지나요?

좋은 답변의 신호

- 재고 확인과 차감의 순서를 코드 위치와 함께 설명한다.
- 재고 0과 초과 요청을 같은 조건으로 처리하는지 구분해 말한다.

우려 신호

- 통과 결과만 말하고 판단 위치를 짚지 못한다.
- 재고 확인 시점을 상황에 따라 다르게 말한다.

근거

R-03 · 재고 부족 거절

<https://ohmyti.vercel.app/evaluations/1bc69b13-00c6-4283-aa1b-52e6f5616362?criterion=R-03>

R-03 재생

<https://ohmyti.vercel.app/evaluations/1bc69b13-00c6-4283-aa1b-52e6f5616362?criterion=R-03&run=ff2fa0ae-631e-4426-bc5b-845e9f3ed740>

src/http/routes.ts:46-67

<https://github.com/wanted-hack-inchyangv/seojin-order-api/blob/d35969b619823452e5edc9bf23ed75dc8f596219/src/http/routes.ts#L46-L67>

src/http/routes.ts:40-44

<https://github.com/wanted-hack-inchyangv/seojin-order-api/blob/d35969b619823452e5edc9bf23ed75dc8f596219/src/http/routes.ts#L40-L44>

R-04 입력 검증이 기대대로 동작했습니다. quantity와 productId를 검사하는 규칙을 어디에 모아 두셨나요?

의도 검증 규칙의 위치와 적용 범위를 스스로 설명하는지 확인한다.

꼬리 질문

1. 검증 실패로 끝난 요청의 키를 기록에 남기지 않으려면 어떤 조건이 필요했나요?
2. JSON이 아닌 본문은 어느 단계에서 걸러지나요?
3. 검증 규칙을 하나 더 추가한다면 어느 파일을 고치시겠어요?

좋은 답변의 신호

- 검증 규칙의 위치와 실패 시 기록 처리 조건을 함께 설명한다.
- 본문 파싱 실패와 필드 검증 실패를 구분해 말한다.

우려 신호

- 검증 항목을 나열만 하고 위치를 말하지 않는다.
- 실패한 요청의 키가 기록에 남는지 여부를 모른다.

근거

R-04 · 입력 검증

<https://ohmyti.vercel.app/evaluations/1bc69b13-00c6-4283-aa1b-52e6f5616362?criterion=R-04>

R-04 재생

<https://ohmyti.vercel.app/evaluations/1bc69b13-00c6-4283-aa1b-52e6f5616362?criterion=R-04&run=a16a7904-6948-4376-ae0e-a6fb9af6c441>

src/http/routes.ts:46-67

<https://github.com/wanted-hack-inchyangv/seojin-order-api/blob/d35969b619823452e5edc9bf23ed75dc8f596219/src/http/routes.ts#L46-L67>

src/http/routes.ts:40-44

<https://github.com/wanted-hack-inchyangv/seojin-order-api/blob/d35969b619823452e5edc9bf23ed75dc8f596219/src/http/routes.ts#L40-L44>

같은 서비스를 여러 인스턴스로 띄워 요청이 서로 다른 인스턴스로 들어오는 상황을 가정하겠습니다. 지금 구조에서 가장 먼저 문제가 드러날 지점은 어디인가요?

의도 인스턴스가 늘어날 때 현재 구조의 한계를 어디로 짚는지 확인한다.

꼬리 질문

1. 그 지점을 먼저 고르신 이유는 무엇인가요?
2. 인스턴스 간에 무엇을 공유해야 같은 키 요청이 한 번만 처리되나요?
3. 공유 저장소를 두면 응답 시간과 실패 처리에서 무엇이 달라지나요?

좋은 답변의 신호

- 인메모리 상태가 인스턴스마다 갈라지는 지점을 구체적으로 짚는다.
- 멍등성 기록과 재고를 어디에 두어야 하는지 구분해 설명한다.

우려 신호

- 인스턴스를 늘리면 된다는 수준에서 멈춘다.
- 공유 저장소 도입 시 생기는 실패 상황을 말하지 않는다.

근거

R-01 · 정상 주문 생성

<https://ohmyti.vercel.app/evaluations/1bc69b13-00c6-4283-aa1b-52e6f5616362?criterion=R-01>

R-05 · 멍등 재전송

<https://ohmyti.vercel.app/evaluations/1bc69b13-00c6-4283-aa1b-52e6f5616362?criterion=R-05>

R-06 · 멍등 키 충돌

<https://ohmyti.vercel.app/evaluations/1bc69b13-00c6-4283-aa1b-52e6f5616362?criterion=R-06>

Q4 **권장** **설계 트레이드오프** 설계와 변경 용이성 · 6분

registerRoutes가 51줄로 가장 긴 함수입니다. 이 함수 안에서 라우팅과 도메인 규칙이 어떻게 나뉘어 있는지 설명해 주시겠어요?

의도 가장 긴 함수의 책임 구성을 스스로 설명하는지 확인한다.

꼬리 질문

1. 이 함수에서 HTTP 요청·응답 객체를 직접 다루는 부분은 어디까지인가요?
2. 재고 차감이나 역등성 판단이 이 함수 밖에 있다면 어느 모듈인가요?
3. 이 함수를 나눈다면 어떤 경계를 먼저 그으시겠어요?

좋은 답변의 신호

- 라우팅과 도메인 규칙의 경계를 코드 위치로 구분해 말한다.
- 도메인 규칙이 요청·응답 객체에 의존하는지 여부를 구체적으로 답한다.

우려 신호

- 함수 길이를 문제로 인식하지 않는다.
- 경계를 말하면서 실제 코드 위치를 짚지 못한다.

근거

src/http/routes.ts:30-80

<https://github.com/wanted-hack-inchyangv/seojin-order-api/blob/d35969b619823452e5edc9bf23ed75dc8f596219/src/http/routes.ts#L30-L80>

R-12 · 설계·변경 용이성

<https://ohmyti.vercel.app/evaluations/1bc69b13-00c6-4283-aa1b-52e6f5616362?criterion=R-12>

Q5 권장 요구사항 확장 트레이드오프 판단과 확장 · 6분

프로세스를 다시 시작해도 주문과 상태가 남아야 하는 상황을 가정하겠습니다. 지금 구조에서 가장 먼저 바꾸실 부분은 어디인가요?

의도 저장 방식이 바뀔 때 구조를 어디부터 손대는지 확인한다.

꼬리 질문

1. 그 부분을 먼저 고르신 이유는 무엇인가요?
2. 저장 위치를 바꾸면 기존 동작 중 무엇을 다시 확인하시겠어요?
3. 초기화 경로는 저장 방식이 바뀌면 어떻게 달라져야 하나요?

좋은 답변의 신호

- 바꿀 범위를 코드의 구체적인 위치로 말한다.
- 저장 위치를 옮길 때 새로 생기는 실패 상황을 든다.

우려 신호

- 도구 이름만 대고 코드에서 바꿀 부분을 말하지 않는다.
- 초기화 경로의 변화를 고려하지 않는다.

근거

R-01 · 정상 주문 생성

<https://ohmyti.vercel.app/evaluations/1bc69b13-00c6-4283-aa1b-52e6f5616362?criterion=R-01>

R-02 · 상품·주문 조회

<https://ohmyti.vercel.app/evaluations/1bc69b13-00c6-4283-aa1b-52e6f5616362?criterion=R-02>

R-05 · 먹등 재전송

<https://ohmyti.vercel.app/evaluations/1bc69b13-00c6-4283-aa1b-52e6f5616362?criterion=R-05>

Q6 권장 이력서 연결 동시성과 데이터 정합성 · 5분

재고 선점 시스템에서 낙관적 잠금 충돌이 났을 때 재시도는 어떤 조건으로 몇 번까지 수행하도록 설계하셨나요?

이력서: TTL 기반 예약과 버전 비교(version check) 기반의 낙관적 잠금(optimistic locking)으로 재고 차감을 직렬화한 결과, 피크 트래픽 구간에서 동시 주문 때문에 생긴 초과 판매를 0건으로 줄였습니다.

의도 이력서의 낙관적 잠금·재시도 설계가 실제로 어떤 충돌 조건과 한도를 다뤘는지 확인한다.

꼬리 질문

1. 버전 비교는 어느 저장소 계층에서 수행되나요?
2. 재시도 사이에 지연이나 백오프를 두었나요?
3. TTL 만료로 회수된 예약과 충돌한 요청은 어떻게 처리되나요?

좋은 답변의 신호

- 충돌 감지 지점과 재시도 횟수·백오프를 구체적으로 설명한다
- TTL 만료 회수와 재시도가 겹치는 경우의 처리 순서를 설명한다

우려 신호

- 재시도를 '충돌하면 다시 시도한다' 수준으로만 설명한다
- 재시도 상한이나 실패 시 응답을 설명하지 않는다

근거

이력서 연결

<https://ohmyti.vercel.app/evaluations/1bc69b13-00c6-4283-aa1b-52e6f5616362?tab=resume#claim-cd8cb39b-e17d-445c-b57f-65b90d7ab12e>

R-08 · 다른 키 동시 요청과 재고 하한

<https://ohmyti.vercel.app/evaluations/1bc69b13-00c6-4283-aa1b-52e6f5616362?criterion=R-08>

Q7 선택 설계 트레이드오프 설계와 변경 용이성 · 6분

R-12 설계 항목을 보겠습니다. 새 오류 코드를 하나 추가한다면 어느 위치를 고치게 되는지 설명해 주시겠어요?

의도 오류 코드와 상태 코드 매핑이 한 곳에 모여 있는지 확인한다.

꼬리 질문

1. 상태 코드와 오류 코드의 매핑은 어디에 정의되어 있나요?
2. 멱등성 기록의 저장과 조회는 어떤 경계 뒤에 있나요?
3. 저장 구현을 바꿀 때 주문 생성 로직에서 고칠 부분이 있나요?

좋은 답변의 신호

- 오류 코드 추가 시 수정할 위치를 구체적으로 짚는다.
- 멱등성 저장소의 경계와 교체 비용을 함께 설명한다.

우려 신호

- 매핑이 여러 곳에 흩어져 있음을 인지하지 못한다.
- 저장소 교체 시 주문 생성 로직 수정 여부를 판단하지 못한다.

근거

R-12 · 설계·변경 용이성

<https://ohmyti.vercel.app/evaluations/1bc69b13-00c6-4283-aa1b-52e6f5616362?criterion=R-12>

Q8 선택 이력서 연결 동시성과 데이터 정합성 · 5분

결제·주문 생성 API에 붙인 멱등성 키 미들웨어에서 같은 키로 진행 중인 요청이 두 번 들어오면 어떻게 병합하도록 구현하셨나요?

이력서: 결제·주문 생성 API에 멱등성 키(Idempotency-Key) 미들웨어를 도입해, 클라이언트 재시도·네트워크 재전송으로 발생하던 중복 주문을 제거했습니다.

의도 이력서의 멱등성 미들웨어가 동시 요청 병합을 어떤 방식으로 보장했는지 확인한다.

꼬리 질문

1. 진행 중 요청의 상태는 어디에 저장했나요?
2. 첫 요청이 실패하면 대기 중이던 요청은 어떤 응답을 받나요?
3. 같은 키에 다른 본문이 오면 어떤 기준으로 충돌을 판정하나요?

좋은 답변의 신호

- in-flight 상태 저장 위치와 병합 시점을 구체적으로 설명한다
- 첫 요청 실패 시 대기 요청의 응답 정책을 설명한다

우려 신호

- 병합을 '락을 건다' 수준으로만 설명한다
- 키 충돌 판정 기준(메서드·경로·본문 해시)을 언급하지 않는다

근거

이력서 연결

<https://ohmyti.vercel.app/evaluations/1bc69b13-00c6-4283-aa1b-52e6f5616362?tab=resume#claim-5147a131-8d53-41e8-890e-3ecaec00e0ae>

R-05 · 멱등 재전송

<https://ohmyti.vercel.app/evaluations/1bc69b13-00c6-4283-aa1b-52e6f5616362?criterion=R-05>

중복 주문으로 인한 결제 취소·환불 건수 0건은 어떤 지표와 기간으로 집계하셨나요?

이력서: 도입 후 3개월간 중복 주문 때문에 생긴 결제 취소·환불 처리 건수가 0건으로 유지되고 있습니다.

의도 수치 주장의 측정 방식과 집계 범위를 확인한다.

꼬리 질문

1. 환불 건수는 어떤 로그나 대시보드에서 집계했나요?
2. 맥등성 키 도입 전후 비교 기준 기간은 어떻게 잡았나요?
3. 키가 없는 클라이언트 요청도 집계에 포함되었나요?

좋은 답변의 신호

- 집계 대상 지표와 데이터 출처를 구체적으로 설명한다
- 도입 전후 비교 기간과 조건을 설명한다

우려 신호

- 수치의 출처를 설명하지 못한다
- 측정 기준 없이 결과만 반복한다

근거**이력서 연결**

<https://ohmyti.vercel.app/evaluations/1bc69b13-00c6-4283-aa1b-52e6f5616362?tab=resume#claim-9852b802-e0a7-4200-88ed-71159fb69b00>

R-06 · 맥등 키 충돌

<https://ohmyti.vercel.app/evaluations/1bc69b13-00c6-4283-aa1b-52e6f5616362?criterion=R-06>

평가 척도 (역량별 4단계)

면접관이 기입할 때 쓰는 기준이며 시스템은 값을 채우지 않습니다.

경계 조건과 실패 처리

잘못된 입력과 자원 부족 같은 실패 경로를 예측하고 일관되게 처리한다.

- | | |
|----------------|---|
| 1 미흡 | 잘못된 입력이나 자원 부족 같은 실패 경로를 떠올리지 못한다. |
| 2 보완 필요 | 실패 경로를 묻으면 떠올리지만, 처리 방식과 응답 형식을 구체적으로 말하지 못한다. |
| 3 충족 | 경계값과 실패 경로를 스스로 나열하고, 각각의 응답 코드와 상태 변화를 설명한다. |
| 4 탁월 | 재시도 중 실패나 부분 실패처럼 실패가 겹치는 상황까지 다루고, 상태가 일관되게 남는 근거를 설명한다. |

동시성과 데이터 정합성

동시에 들어오는 요청에서도 데이터가 어긋나지 않도록 보호한다.

- | | |
|----------------|---|
| 1 미흡 | 동시에 들어온 요청 때문에 상태가 어긋날 수 있다는 점을 인식하지 못한다. |
| 2 보완 필요 | 경쟁 조건이 생길 수 있다는 점은 알지만, 어느 구간에서 생기는지 짚지 못한다. |
| 3 충족 | 경쟁이 생기는 구간과 그 구간을 보호하는 방법(락, 원자적 연산, 유일성 제약)을 코드 위치와 함께 설명한다. |
| 4 탁월 | 저장소가 바뀌거나 인스턴스가 늘어날 때 보호 방법이 깨지는 조건을 짚고, 대안을 비교해 설명한다. |

설계와 변경 용이성

책임을 나누어 요구가 바뀌었을 때 고칠 곳이 좁고 분명한 구조를 만든다.

- 1 미흡 코드 구조를 그렇게 나눈 이유를 설명하지 못한다.
- 2 보완 필요 구조는 설명하지만, 요구가 바뀌면 어디를 고쳐야 하는지 분명히 말하지 못한다.
- 3 충족 책임을 나눈 기준을 설명하고, 요구 변경 하나에 대해 고칠 위치를 코드에서 짚는다.
- 4 탁월 현재 구조의 한계를 스스로 짚고, 다른 구조와 비교해 선택에 따르는 비용을 설명한다.

트레이드오프 판단과 확장 면접에서만 확인

선택지의 비용과 이점을 기준을 세워 비교하고 조건이 바뀌면 판단을 조정한다.

- 1 미흡 선택지 사이의 차이를 설명하지 못하고 한 가지 방법만 제시한다.
- 2 보완 필요 선택지를 나열하지만, 무엇을 기준으로 골라야 하는지 말하지 못한다.
- 3 충족 정합성, 성능, 복잡도 같은 판단 기준을 밝히고, 과제 조건에서 그 방법을 고른 이유를 설명한다.
- 4 탁월 트래픽 증가나 다중 인스턴스처럼 조건이 바뀌면 선택이 어떻게 달라지는지 근거와 함께 설명한다.

운영과 관측 가능성 면접에서만 확인

배포 뒤에 문제를 알아차리고 영향을 확인해 복구할 수 있게 만든다.

- 1 미흡 배포한 뒤에 문제가 생기면 어떻게 알아차릴지 설명하지 못한다.
- 2 보완 필요 로그를 남긴다고 답하지만, 무엇을 기록하고 어떻게 찾을지는 구체적으로 말하지 못한다.
- 3 충족 문제를 알아차릴 신호(로그, 지표, 상태 확인)와 확인 순서를 구체적으로 설명한다.
- 4 탁월 운영 중에 데이터가 어긋났을 때 영향 범위를 확인하고 복구하는 절차까지 설명한다.